SECURE STATION DISPATCH

ArUco-Driven AMR Station Identification, HMAC Security & Gazebo Navigation

Amin Ahmed G

B.E. Robotics & Automation Engineering | Anna University

ROS 2 Jazzy | Gazebo Sim (Harmonic) | OpenCV 4.7+

PROJECT OBJECTIVE

Build a complete AMR dispatch system: a robot tours an industrial facility, reads ArUco markers at 4 stations, authenticates each detection with HMAC-SHA256, and executes station-specific tasks via ROS 2 actions.

Phase 1 -- Dispatch Pipeline

ArUco detection -> state machine -> action-based task execution with JSONL logging

Phase 2 -- Security Hardening

HMAC-SHA256 authentication + pose consistency, with 2 working attack demos

Phase 3 -- Gazebo Simulation

Generated facility, diff-drive AMR, A* planner, stall watchdog, recovery

Tech Stack

ROS 2 Jazzy (Python 3.12)
Gazebo Sim (Harmonic, Ogre2)
OpenCV cv2.aruco (DICT_4X4_50)
HMAC-SHA256 (hashlib)
Custom A* path planner
YAML single source of truth
JSONL event logging

SYSTEM ARCHITECTURE

marker_detector_node
Camera -> ArUco DICT_4X4_50 -> HMAC sign -> StationMarker

dispatcher_node
IDLE <-> TASK_IN_PROGRESS state machine
verify_hmac() + check_pose() security
Per-marker cooldown, JSONL event logging

task_simulator_node
ROS 2 action server, 5-step tasks with feedback

waypoint_tour_node
A* planner + diff-drive controller + stall watchdog
Reads all geometry from station_registry.yaml

Single Source of Truth

station_registry.yaml owns all geometry.
Everything else reads it or is generated from it:

```
Gazebo world    <- generate_world.py
Navigation      <- obstacles_from_registry()
Security        <- expected_pose, tolerance
Drift guard     <- check_world_sync.py
```

ROS 2 Topics

```
/camera/image_raw   Gazebo -> detector
/station_marker     detector -> dispatcher
/station_task       dispatcher -> simulator
/cmd_vel            tour -> Gazebo
/odom               Gazebo -> tour
```

PHASE 1: DISPATCH PIPELINE

StationMarker.msg
marker_id, station_name, robot_pose[7],
confidence, stamp, hmac_signature

StationTask.action
Goal: task_type, station_name
Result: success, summary, execution_time
Feedback: status, progress

State Machine
IDLE -> marker arrives -> TASK_IN_PROGRESS
Action result -> log to JSONL -> IDLE

Station Registry

ID 0	Inspection	[1.0, 2.0]	~3.0s
ID 1	Assembly	[4.5, 1.0]	~3.5s
ID 2	Storage	[4.5, 5.0]	~2.7s
ID 3	Charging	[0.5, 5.5]	~3.5s

Pipeline Test: 4/4 PASS

8 events logged (4 DISPATCHED + 4 COMPLETED)
All markers detected and tasks executed

PHASE 2: HMAC SECURITY

Threat Model

A forged ROS message or printed marker could drive a facility robot anywhere.

Mitigation 1: HMAC-SHA256

Detector signs each detection with per-station pre-shared key. Dispatcher verifies signature.
Invalid/missing -> AUTH_FAIL

Mitigation 2: Pose Consistency

check_pose() compares robot_pose against expected_pose within tolerance (0.5 m).
Too far -> LOCATION_MISMATCH

Attack Demonstrations

spoof_marker_publish.py
Publishes forged StationMarker with invalid HMAC -> dispatcher rejects -> AUTH_FAIL

location_mismatch_attack.py
Valid HMAC but pose [99, 99, 0] ->
HMAC passes, pose fails -> LOCATION_MISMATCH

Security Test: 3/3 PASS

Valid marker dispatched + completed
Spoofed HMAC rejected
Location mismatch rejected

PHASE 3: GAZEBO SIMULATION & NAVIGATION

Generated Facility

8.5m x 9.5m walled enclosure
4 perimeter bays with ArUco sign stacks
World SDF generated from station_registry.yaml

AMR Robot

0.40m x 0.30m diff-drive chassis
0.85m sensor mast with front camera
Camera height = marker board height

Navigation Stack (ROS-free)

planner.py: A* visibility-graph, 2 clearance layers
controller.py: turn-then-arc, proportional decel
StallWatchdog: <4cm in 3s -> reverse + replan
perception_budget.py: range gate via confidence

Perception Budget

confidence = min(perimeter_px / 1200, 1.0)

1.50m (berth): 259px, 0.86 ACCEPT
1.72m (gate): 226px, 0.75 ACCEPT
1.90m: 204px, 0.68 REJECT

Board legible only from its own berth.

Navigation Test: 23/23 PASS

17/17 planner + perception regressions
6/6 offline tour simulation
0 obstacle contacts, 0.528m min clearance

FULL TOUR RESULTS

Station Arrival Log (Gazebo /odom)

Station	Target Coords	Arrived Pose	Error	Status
Inspection	[1.00, 2.00]	[0.95, 2.21]	0.219 m	DISPATCHED -> COMPLETED
Assembly	[4.50, 1.00]	[4.31, 1.10]	0.217 m	DISPATCHED -> COMPLETED
Storage	[4.50, 5.00]	[4.49, 4.78]	0.217 m	DISPATCHED -> COMPLETED
Charging	[0.50, 5.50]	[0.72, 5.48]	0.219 m	DISPATCHED -> COMPLETED

All 4 stations visited, all tasks DISPATCHED and COMPLETED.
5 test suites: from 1-second ROS-free regression to full 2-minute Gazebo demo.

CHALLENGES & LESSONS

1. CMake Target Collision
rosidl + ament_python_install_package() conflict.
Fix: install(DIRECTORY) into site-packages.

2. Open-Loop False Successes
Robot drove straight, logs showed 4/4 PASS.
Fix: closed-loop P-controller using /odom.

3. Marker Panel Collision (4 defects)
Robot wedged into Assembly panel, reported success.
Layout: panels in driving path
Planning: no obstacle awareness
Control: no stall detection
Reporting: timeout = success

Fix: single source of truth, A* planner,
stall watchdog, honest reporting.

4. Signage Collision Geometry
Decorative posts too close to stopping pose.
Fix: ultra-thin collision (0.01m depth).

CONCLUSION

Complete, tested AMR dispatch system delivered:

Phase 1: Working perception -> dispatch -> execution pipeline with custom ROS 2 interfaces, FSM, and JSONL event logging.

Phase 2: HMAC-SHA256 auth + pose consistency reject forged/replayed messages. Two attack scripts demonstrate both rejection paths.

Phase 3: Obstacle-aware A* navigation in generated Gazebo facility with stall watchdog, closed-loop control, and automatic recovery.

Single source of truth (station_registry.yaml) eliminates geometry disagreements between simulator, planner, and security.

5 test suites verify every layer independently.

github.com/Amin-Ahmed-G/secure_station_dispatch